Code Generation for a One-Register Machine

JOHN BRUNO AND RAVI SETHI

The Pennsylvania State University, University Park, Pennsylvania

ABSTRACT. The majority of computers that have been built have performed all computations in devices called accumulators, or registers. In this paper, it is shown that the problem of generating minimal-length code for such machines is hard in a precise sense; specifically it is shown that the problem is NP-complete. The result is true even when the programs being translated are arithmetic expressions. Admittedly, the expressions in question can become complicated.

KEY WORDS AND PHRASES: register allocation, straight line programs, basic blocks, arithmetic expressions, NP-complete, code optimization

CR CATEGORIES: 4.12, 5.25

1. *Introduction*

Starting with the work of Anderson for a one-accumulator machine [4], there is a considerable body of published literature on code generation for arithmetic expressions. It is usual to represent expressions like $(a + b)/(b - c)$ and $(b - c)*d$ by trees as in Figure 1(a). Leaves in the tree represent initial values, and nonleaf nodes represent the results of operations. Aho and Johnson [2] show that for a wide range of machines, code generation for trees is a manageable problem; algorithms tend to take time linear in the size of the tree. Previous work on the problem may be found in [4–7, 9, 10, 13–15].

Consider the expressions in Figure 1(a). If both expression trees are computed, then $b - c$ will be computed twice. The graph (Figure 1(b)) formed by collapsing identical subtrees exhibits all the common subexpressions in Figure 1(a). Such a graph representation is a directed acyclic graph (dag). In fact, dags conveniently represent sequences of assignment instructions [3].

In this paper we will show that code generation for dags is difficult, even on a very simple machine. We will consider a machine with a single register (accumulator) and three kinds of instructions: (a) *load*, (b) *store*, and (c) *operate*. $A \leftarrow B + C$ is computed by loading the value B into the register, performing the operation $ADD\ C$, and then storing the result into A. Expressions like $B*C + D*E$ can be computed by computing $D*E$ into the register, storing it in a temporary T, computing $B*C$ into the register, and then executing an $ADD\ T$ instruction. The *length* of a program for computing an expression is the number of loads, operations, and stores in the program.

Recent work in computational complexity has related the inherent difficulty of a large class of combinatorial problems. Members of this class are referred to as *NP-complete* problems and include the traveling salesman and graph coloring problems. Informally, in order to show that a new problem A is NP-complete, it has to be shown that A is as difficult as a known NP-complete problem but not too difficult, i.e. $A \in NP$. The reader is referred to the textbook [1] for a discussion of this class of problems.

The main result of this paper is that determining a minimal length program for a dag

Authors' address: Computer Science Department, The Pennsylvania State University, University Park, Pennsylvania 16802.

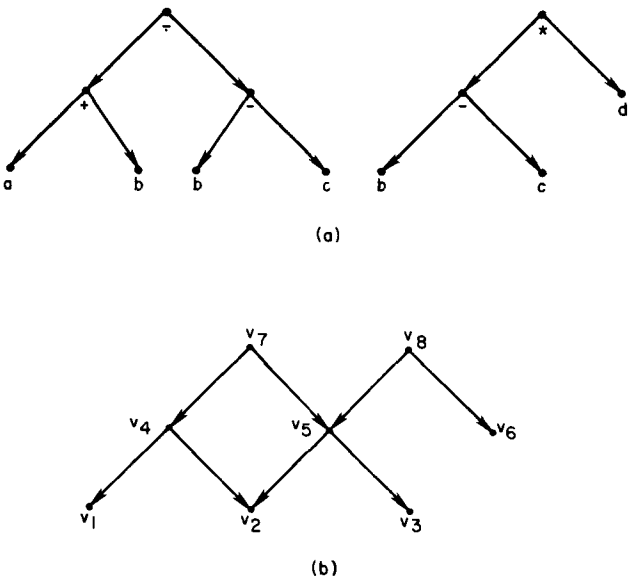

FIG. 1 (a) Trees for the expressions $(a + b)/(b - c)$ and $(b - c)*d$. (b) The graph formed from the trees in (a) by collapsing the common subexpression $b - c$ and identifying all the nodes for b.

omputed on the one-register machine given above is an NP-complete problem. In udging the applicability of this result, it might be noted that the simple machine we have chosen corresponds to a variety of computers, ranging from hand calculators to the IBM 709.

2. *The Model*

Expressions will be represented by dags. For our purposes, each vertex in a dag has either zero or two descendants. As in Figure 1(b) a vertex with zero descendants represents an initial value and a vertex with two descendants corresponds to a binary operator. Since with binary operators like "$-$", $a - b$ and $b - a$ are not the same, we will order the descendants of each vertex. One descendant is called the *left* descendant, and the other the *right* descendant. A vertex with no descendants is called a *leaf* and a vertex with no ancestors a *root*.

In order to talk about programs that compute dags, we need the notion of "value" of a dag. We will assume that there are no algebraic properties like distributivity or associativity between operators. The reason for this assumption is that for many expressions these laws are not applicable. Moreover, for the expressions we will need to consider the operators can be chosen to make algebraic simplifications not readily applicable. We will therefore talk in terms of an anonymous binary operator θ that has no algebraic properties.

If D is a dag and v is a vertex of D, then the *value* of v, denoted $val(v)$, is defined as follows:

1. If v is a leaf, then $val(v) = v$.
2. If v is not a leaf and v_1 and v_2 are the left and right descendants, respectively, of v, then $val(v) = (val(v_1)\ \theta\ val(v_2))$.

Example 1. Let D be the dag depicted in Figure 1(b). Below we calculate some values of the vertices of D.

$$val(v_3) = (val(v_5)\ \theta\ val(v_6)) = ((val(v_2)\ \theta\ val(v_3))\ \theta\ val(v_6)) = ((v_2\ \theta\ v_3)\ \theta\ v_6).$$

$$val(v_4) = (val(v_1)\ \theta\ val(v_2)) = (v_1\ \theta\ v_2).$$ □

We assume a single-accumulator machine with LOAD, STORE, and OPERATE

instructions. Let ACC denote the contents of the accumulator. In addition there is an infinite capacity main memory; the contents of location α, where α is some positive integer, are denoted by $C(\alpha)$.

Let α be some positive integer. The *machine instructions* are:

1. $LD\ \alpha$ (means $ACC \leftarrow C(\alpha)$)
2. $ST\ \alpha$ (means $C(\alpha) \leftarrow ACC$)
3. $OP\ \alpha$ (means $ACC \leftarrow (ACC\ \theta\ C(\alpha))$).

If D is a dag to be "evaluated" by our machine, we shall assume that the values of all the leaves of D are stored in unique locations in main memory.

A *program* is a sequence of machine instructions. A program P is said to *evaluate* a dag D if after carrying out the operations indicated by P, the values of all the roots of D occupy unique locations in main memory.

Example 2. Let D be the dag in Figure 1(b). The initial contents of main memory are given by:

Main memory locations	Initial values
1	v_1
2	v_2
3	v_3
4	v_6
location ≥ 5	— (don't care)

Below is the list of instructions for a program which evaluates D; to the right of each instruction is the value of ACC after the instruction is carried out.

Program	ACC	Program	ACC
$LD\ 2$	v_2	$LD\ 1$	v_1
$OP\ 3$	$(v_2\ \theta\ v_3)$	$OP\ 2$	$(v_1\ \theta\ v_2)$
$ST\ 5$	$(v_2\ \theta\ v_3)$	$OP\ 5$	$((v_1\ \theta\ v_2)\ \theta\ (v_2\ \theta\ v_3))$
$OP\ 4$	$((v_2\ \theta\ v_3)\ \theta\ v_6)$	$ST\ 7$	$((v_1\ \theta\ v_2)\ \theta\ (v_2\ \theta\ v_3))$
$ST\ 6$	$((v_2\ \theta\ v_3)\ \theta\ v_6)$		

The above nine-instruction program determines $val(v_7)$ and $val(v_8)$ and stores these values in locations 7 and 6, respectively. □

Given a dag D, we are motivated to find a shortest program which evaluates D. We shall investigate the complexity of this problem by considering, in Section 3, the complexity of a related problem called $EVAL$. Subsequently, we shall return to the question of finding a shortest length program.

3. *A Reduction*

We define problem $EVAL$ as follows.

Problem EVAL
Given: A dag D, and a positive integer q.
Question: Does there exist a program of length at most q which evaluates D?

A solution to $EVAL$ would be an algorithm which takes as input a dag D and a positive integer q and outputs the answer to the question (i.e. either true or false). At this point we introduce problem $SAT3$, which is crucial to demonstrating that $EVAL$ is a hard problem.

Let n be a positive integer and $X_n = \{x_1,\bar{x}_1,\cdots,x_n,\bar{x}_n\}$. The elements of X_n are called *literals*. The *complement* of $x_i(\bar{x}_i)$ is $\bar{x}_i(x_i)$.

Informally, a literal y in X_n can be either true or false. If y is true then the complement of y is false, and vice versa. In defining $SAT3$ we will define "clauses" C_j, like x_1 *or* x_2 *or* $\bar{x}_3$. A clause is true if one of the literals in it is true. Given a set of clauses $C_1, C_2, \cdots, C_m$, the clauses are "satisfiable" if under some truth assignment to literals in X_n, all

the clauses are true. In the definition of $SAT3$ the set K will specify exactly which literals in X_n are true. In order for a truth assignment K to "satisfy" $C_1, C_2, \cdots, C_m$ for each C_j, one of the literals in K must also be in C_j, i.e. $K \cap C_j \neq \varnothing$.

Problem SAT3

Given: $Q = \langle n, C_1, \cdots, C_m \rangle$, where n and m are positive integers, $n \leq 3m$, $C_j \subseteq X_n$, and $|C_j| = 3$ for $j = 1, \cdots, m$.

Question: Does there exist a set $K = \{y_1, \cdots, y_n\}$ such that y_i equals either x_i or $\bar{x}_i$ for $i = 1, \cdots, n$ and $K \cap C_j \neq \varnothing$ for $j = 1, \cdots, m$? If there is such a set K for a given Q, we say that Q is *satisfiable.*

A solution to $SAT3$ would be an algorithm which takes an instance Q of $SAT3$ and outputs true if and only if Q is satisfiable. From [8] it follows that $SAT3$ is NP-complete, or informally, $SAT3$ is known to be a hard problem.

We will show that problem $EVAL$ is at least as difficult as problem $SAT3$. We can accomplish this objective by providing an algorithm A that takes as input an instance, Q, of $SAT3$, and produces an instance $D(Q)$ and a positive integer q, of $EVAL$. There will be a program of length no more than q which evaluates $D(Q)$ if and only if Q is satisfiable. If $Q = \langle n, C_1, \cdots, C_m \rangle$, let the *size* of Q be $n + m$. We will show that $D(Q)$ has N nodes, where for some constant c, $N \leq (n + m)^c$.

Suppose for some constant d we can answer problem $EVAL$ in N^d steps. Then given $D(Q)$, since there is a program of length at most q for $D(Q)$ if and only if Q is satisfiable, by solving problem $EVAL$ we can determine in $(n + m)^{c \cdot d}$ steps if Q is satisfiable. Since $c \cdot d$ is a new constant, $(n + m)^{c \cdot d}$ is a polynomial in $n + m$, the size of Q.

Suppose algorithm A also takes a polynomial number of steps in $n + m$ to determine $D(Q)$ from Q. Then, starting with Q we can determine whether Q is satisfiable in a polynomial number of steps in $n + m$ *if there is an algorithm to answer problem EVAL in a polynomial number of steps.* Recall that we suspect that $SAT3$ cannot be solved in a polynomial number of steps in $n + m$. We must therefore suspect that $EVAL$ cannot be solved in a polynomial number of steps in N, the number of nodes in a dag.

In the preceding paragraphs we have given an informal treatment of why we expect that problem $EVAL$ is difficult. A more rigorous treatment of NP-complete problems may be found in [1, 8, 11, 12].

Our goal is to prove that there is a polynomial-time algorithm which accepts as input an instance, Q, of $SAT3$ and produces an instance, $D(Q)$ and q, of $EVAL$ such that there exists a program of length q which evaluates $D(Q)$ if and only if Q is satisfiable.

Let $Q = \langle n, C_1, \cdots, C_m \rangle$ be an instance of $SAT3$. We shall show how to construct the dag $D(Q)$. The construction is given in Figures 2 and 3. We denote the elements of C_j as y_{1j}, y_{2j}, and y_{3j}, i.e. $C_j = \{y_{1j}, y_{2j}, y_{3j}\}$ for $j = 1, \cdots, m$. In Figure 2 we show the subgraph of $D(Q)$, called the *trunk*, and in Figure 3 we show $m + n$ additional pieces (subgraphs) of $D(Q)$. Intuitively, the subgraph L_i represents the literals x_i and $\bar{x}_i$, for

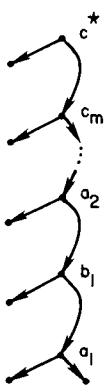

FIG. 2. The trunk of $D(Q)$ is a tree in which the nonleaf nodes form a chain. Proceeding toward the root, the labels of nodes on the chain are $a_1, b_1, a_2, b_2, \cdots, a_n, b_n, c_1, c_2, \cdots, c_m, c^*$.

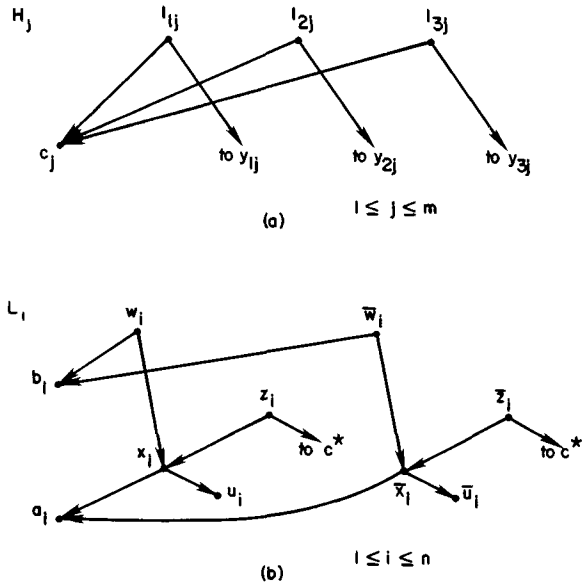

FIG. 3. Sections of $D(Q)$.

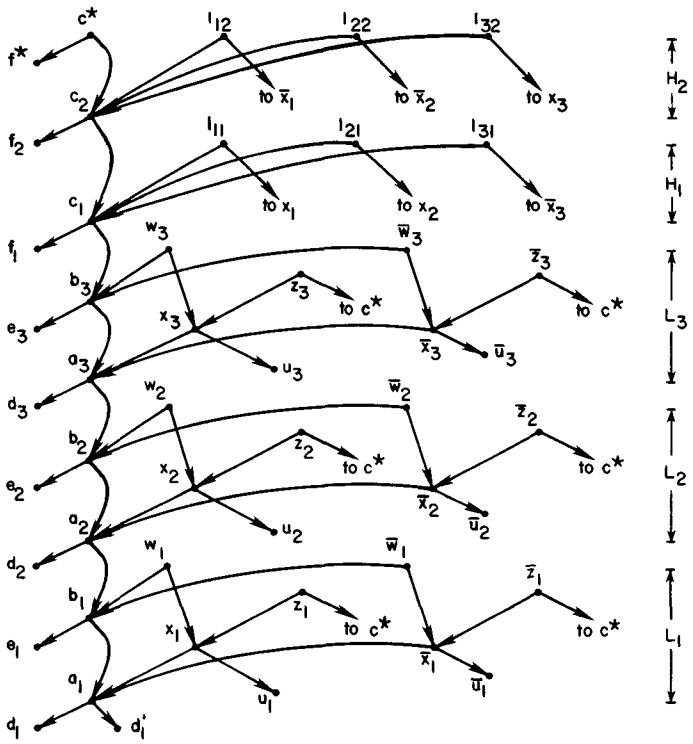

FIG. 4 The dag $D(Q)$ for the clauses $\{\bar{x}_1, x_2, \bar{x}_3\}$ and $\{\bar{x}_1, \bar{x}_2, x_3\}$. For readability some of the edges, such as the one from l_{31} to $\bar{x}_3$, have not been completed.

$i = 1, \cdots, n$. The arcs from the vertices in H_j to the vertices y_{1j}, y_{2j}, and y_{3j} represent the set C_j for $j = 1, \cdots, m$.

Example 3. Let $Q = \{3, C_1, C_2\}$, where $C_1 = \{x_1, x_2, \bar{x}_3\}$ and $C_2 = \{\bar{x}_1, \bar{x}_2, x_3\}$. The dag $D(Q)$ is shown in Figure 4. In order to avoid drawing a diagram complicated by too

many connections, we have not extended every arc to its destination but, instead, have given the destination as a label. For example, none of the arcs entering c^* and none of the arcs from subgraphs H_1 and H_2 to the subgraphs L_1, L_2, and L_3 have been completed.

In this example, $n = 3$ and $m = 2$ in the specification of Q. We will show that $21n + 11m + 3$ instructions are enough to compute $D(Q)$. To start with, note that if x_1 and x_3 are true and x_2 is false, then both C_1 and C_2 are true. As a consequence, we will compute x_1 before $\bar{x}_1$, $\bar{x}_2$ before x_2, and x_3 before $\bar{x}_3$ in parts L_1, L_2, and L_3, respectively, of $D(Q)$.

	Program	*ACC*	Comments
1	*LD* d_1	d_1	
2	*OP* d_1'	a_1	
3	*ST* a_1	a_1	We will need a_1 later
4	*OP* u_1	x_1	We chose x_1 instead of $\bar{x}_1$.
5	*ST* x_1	x_1	
6	*LD* e_1	e_1	
7	*OP* a_1	b_1	
8	*ST* b_1	b_1	
9	*OP* x_1	w_1	Since x_1 was computed we can use it.
10	*ST* w_1	w_1	
⋮	⋮	⋮	The next 20 instructions compute a_2, $\bar{x}_2$, b_2, $\bar{w}_2$, a_3, x_3, b_3, and w_3. The program so far contains $10n$ instructions
31	*LD* f_1	f_1	
32	*OP* b_3	c_1	
33	*ST* c_1	c_1	
34	*OP* x_1	l_{11}	Since x_1 was computed we can use it
35	*ST* l_{11}	l_{11}	
	⋮	⋮	The next 5 instructions compute c_2 and l_{22}. The program so far contains $10n + 5m$ instructions
41	*LD* f^*	f^*	
42	*OP* c_2	c^*	
43	*ST* c^*	c^*	With the last 3 instructions the total becomes $10n + 5m + 3$.
44	*LD* a_1	a_1	We now compute the remaining nodes in L_1.
45	*OP* $\bar{u}_1$	$\bar{x}_1$	
46	*ST* $\bar{x}_1$	$\bar{x}_1$	
47	*OP* c^*	$\bar{z}_1$	
48	*ST* $\bar{z}_1$	$\bar{z}_1$	
49	*LD* x_1	x_1	
50	*OP* c^*	z_1	
51	*ST* z_1	z_1	
52	*LD* b_1	b_1	
53	*OP* $\bar{x}_1$	$\bar{w}_1$	
54	*ST* $\bar{w}_1$	$\bar{w}_1$	
⋮	⋮	⋮	The next 22 instructions compute the remaining nodes in L_2 and L_3 Total $21n + 5m + 3$
77	*LD* c_1	c_1	We now compute the remaining nodes in H_1.
78	*OP* x_2	l_{21}	Note that x_2 has now been computed.
79	*ST* l_{21}	l_{21}	
80	*LD* c_1	c_1	
81	*OP* $\bar{x}_3$	l_{31}	
82	*ST* l_{31}	l_{31}	
⋮	⋮	⋮	The next 6 instructions compute the remaining nodes in H_2 Total $21n + 11m + 3$.

Points to be noted:

(a) When instruction 3, *ST* a_1, is executed, the value of a_1 is still in the register *ACC*. Thus we can immediately compute one of x_1 and $\bar{x}_1$. If neither x_1 nor $\bar{x}_1$ is computed now, then we will later need to load a_1 for both x_1 and $\bar{x}_1$ instead of just for one of them.

(b) When instruction 33, *ST* c_1, is executed, the value of c_1 is still in the register

ACC. From point (a) above, we expect that in subgraph L_i, $1 \leq i \leq n$, x_i or $\bar{x}_i$ has been computed. If one of the literals in clause C_1 has been computed, then one of l_{11}, l_{21}, and l_{31} can be computed using the value of c_1 in *ACC*. Otherwise an extra load instruction will later be required. □

THEOREM. *Let* $Q = \langle n, C_1, \cdots, C_m \rangle$ *be an instance of SAT*3. *There exists a program of length* $21n + 11m + 3$ *which evaluates* $D(Q)$ *if and only if* Q *is satisfiable.*

The proof of this theorem is given in the Appendix.

A readable introduction to NP-complete problems can be found in [1]. Here we merely note that $D(Q)$ can be constructed from Q in time linear in $n + m$. Given the above theorem, it is routine to verify that problem *EVAL* is NP-complete.

4. *Complexity Implications*

In Section 3 we were concerned with problem *EVAL*, which determines if a dag D can be computed using at most q instructions. In practice we are interested in constructing shortest programs for D, not in whether q instructions will suffice. We will now informally explore the relationship between *EVAL* and the construction of a shortest program for D.

Suppose there is a polynomial time algorithm A for constructing shortest programs. A can immediately be used to solve *EVAL* by constructing a shortest program and checking if it has more than q instructions. From our theorem we can then solve *SAT*3 in polynomial time; this is an unlikely event.

It is not as readily apparent that a polynomial algorithm for solving *EVAL* will enable us to construct shortest programs. Given a dag D with N nodes, D can be computed by a program of length at most $3N$ as follows: For each node x in D, load the left descendant of x (when available), perform the operation that computes x using the right operand from storage, and then store x. We can use *EVAL* repeatedly to determine the length q^* of a shortest program by checking if $3N-1$, $3N-2$, $\cdots$ instructions are enough. Since each execution of *EVAL* takes polynomial time in N, the time expenditure so far is polynomial in N.

The basic idea now is to construct the shortest program instruction by instruction. The first instruction must be the load of some left descendant leaf. There can be at most N of these leaves. For each such leaf x, we can use a variant of problem *EVAL* to determine if, starting with x in the register, the dag can be computed using $q^* - 1$ instructions. One uses the same technique to next decide which *OP* α instruction is to be done. A similar approach was used by Sahni [11] in determining the minimal equivalent graph of a digraph.

The above approach will take time polynomial in N only if there exists a polynomial-time algorithm for *EVAL*. The following remarks indicate why this event is unlikely.

The class of languages accepted by (non)deterministic Turing machines in polynomial time (in the size of the input) is denoted by $[NP]P$. A major open question in complexity theory is whether $P = NP$. Cook [8] has shown that there exists a polynomial-time algorithm for *SAT*3 if and only if $P = NP$. From our theorem relating *EVAL* and *SAT*3, there is a polynomial time algorithm for *EVAL* if and only if $P = NP$.[1] Since it is widely suspected that $P \neq NP$, we stated earlier that a polynomial time algorithm for *EVAL* or *SAT*3 is unlikely.

From the discussion relating *EVAL* and the problem of constructing a shortest program, similar comments apply to the latter problem.

5. *Multiregister Machines*

The discussion in the last few sections has been limited to a particular one-register machine. The reason for this choice is that a one-register machine is the simplest case of a

[1] In fact, from the definition of NP-complete problems, there is a polynomial algorithm for *any* NP-complete problem if and only if $P = NP$.

machine with N general registers. Moreover, the one-register machine is also the simplest example of a stack machine like a Burroughs B5500 with a stack of depth N. If generating code for a one-register machine is difficult, then we expect that code generation for any reasonable architecture that subsumes a one-register machine to be difficult as well.

With multiregister machines another cost criterion is of interest. Since most expressions need a small number of temporary locations, instead of storing intermediate values into memory, they can be held in registers. The problem of evaluating a dag then becomes one of determining the minimal number of registers for computing the dag without stores. This problem has been shown to be NP-complete in [12].

The questions of interest at this point are about the inherent properties of dags and machines that make code generation difficult. We know that trees can be handled easily [2]. Are there subclasses of dags for which code generation can be done in polynomial time?

Appendix

Our aim is to show that $D(Q)$ can be evaluated by a program of length $21n + 11m + 3$ if and only if Q is satisfiable. We do this by counting the number of instructions required by any program which evaluates $D(Q)$ and point out where alternatives, and thus savings, exist.

If P is a program which evaluates $D(Q)$, then P determines a linear order $<_P$ on the nonleaf nodes of $D(Q)$, namely $v_1 <_P v_2$ if $val(v_1)$ is evaluated (obtained in ACC) before $val(v_2)$ according to P. For example, the program given earlier to evaluate the expression in Figure 1(b) determines the relations $v_5 < v_3 < v_4 < v_7$.

It is clear that if P is any program which evaluates $D(Q)$, then

(i) $a_i <_P b_i$, $1 \le i \le n$,
(ii) $b_i <_P a_{i+1}$, $1 \le i \le n-1$,
(iii) $b_n <_P c_1$,
(iv) $c_i <_P c_{i+1}$, $1 \le i \le m-1$,
(v) $c_m <_P c^*$,
(vi) $c^* <_P z_i$ and $c^* <_P \bar{z}_i$, $1 \le i \le n$.

Let us count the number of instructions which are absolutely necessary for any program which evaluates $D(Q)$.

The evaluation of a_i requires that we load its left operand, perform the operation, and then store the result (since it must be used as a right operand later on). A similar remark applies to b_i. This accounts for $6n$ instructions. Similar considerations for the c_j's and c^* account for an additional $3m + 3$.

Consideration of the six vertices w_i, $\bar{w}_i$, x_i, $\bar{x}_i$, z_i, and $\bar{z}_i$ yields $12n$ additional instructions when we take into account an operation and a store instruction for each. The vertices l_{1j}, l_{2j}, and l_{3j} account for an additional $6m$ instructions.

We have thus far accumulated $18n + 9m + 3$ instructions which must appear in any program which evaluates $D(Q)$. Up to this point we have ignored the possible load instructions needed for the left operands of x_i, $\bar{x}_i$; z_i, $\bar{z}_i$; w_i, $\bar{w}_i$, and l_{1j}, l_{2j}, and l_{3j}. We can save on load instructions if we evaluate some of the above vertices when the left operand appears in the accumulator for the first time.

LEMMA 1. *Assume Q is satisfiable. Then $D(Q)$ can be evaluated by a program with $21n + 11m + 3$ instructions.*

PROOF. Similar to Example 3.

LEMMA 2. *Let P be a program which evaluates $D(Q)$. If for some x_i either $x_i <_P c^*$ and $\bar{x}_i <_P c^*$ or $c^* <_P x_i$ and $c^* <_P \bar{x}_i$, then the number of instructions in P is greater than $21n + 11m + 3$.*

PROOF. In each section L_i there is the potential for 6 load instructions (2 for a_i, 2 for b_i, and 1 each for x_i and $\bar{x}_i$). If x_i and $\bar{x}_i <_P c^*$, then both x_i and $\bar{x}_i$ will require a load instruction, thereby bringing the minimum number of load instructions to 4 (since there must be at least one load for each of a_i and b_i).

If $c^* <_P x_i$ and $\bar{x}_i$, then we will require 2 load instructions for each of a_i and b_i bring-

ing the minimum number of load instructions to 4. Since there must be at least 2 load instructions for each c_j, we find that the total number of instructions in P must be greater than $21n + 11m + 3$. □

LEMMA 3. *Let P be a program which evaluates $D(Q)$. If for some c_j we have $c_j <_P y_{1_j}$, $c_j <_P y_{2_j}$, and $c_j <_P y_{3_j}$, then P contains more than $21n + 11m + 3$ instructions.*

PROOF. Clearly if the hypothesis of the lemma is true, then c_j will be loaded three times: once each for l_{1_j}, l_{2_j}, and l_{3_j}. Assuming all other c's use the minimum of 2 load instructions and the vertices in L_i account for a minimum of 3 load instructions, we find that the number of instructions in P must be greater than $21n + 11m + 3$. □

LEMMA 4. *If P is a program with exactly $21n + 11m + 3$ instructions which evaluates $D(Q)$, then Q is satisfiable.*

PROOF. By Lemma 2 we conclude that there is a set $K = \{y_1, \cdots, y_n\}$ such that y_i is either x_i or $\bar{x}_i$ and $y_i <_P c^* <_P \bar{y}_i$ ($\bar{y}_i$ is the complement of y_i). By Lemma 3 we conclude that there exists a k_j such that $y_{k_j} <_P c_j$. Since $c_j <_P c^*$ it follows that $y_{k_j} \in K$, and thus $K \cap C_j \neq \varnothing$ for $j = 1, \cdots, m$. □

ACKNOWLEDGMENTS. The comments of the referees on the presentation of our results have helped us greatly in revising the paper.

REFERENCES

1 AHO, A.V., HOPCROFT, J.E., AND ULLMAN, J D. *Design and Analysis of Computer Algorithms.* Addison-Wesley, Reading, Mass., 1974.
2. AHO, A V , AND JOHNSON, S C. Optimal code generation for expression trees. Seventh Annual ACM Symposium on Theory of Computing, May 1975, pp. 207–217.
3. AHO, A.V., AND ULLMAN, J.D. Optimization of straight line programs *SIAM J. Comput. 1*, 1 (March 1972), 1–19.
4. ANDERSON, J P A note on some compiling algorithms *Comm. ACM 7*, 3 (March 1964), 149–150
5 BEATTY, J.C. An axiomatic approach to code optimization for expressions. *J. ACM 19*, 4 (Oct 1972), 613–640; Errata *20*, 1 (Jan. 1973), 188, and *20*, 3 (July 1973), 538.
6 BRUNO, J , AND LASSAGNE, T. The generation of optimal code for stack machines *J. ACM 22*, 3 (July 1975), 382–396.
7. CHROUST, G. Scope conserving expression evaluation. Proc. IFIP Cong. 1971, North-Holland Pub. Co , Amsterdam, pp 178–182.
8. COOK, S.A. The complexity of theorem-proving procedures. Third Annual ACM Symposium on Theory of Computing, May 1971, pp 151–158.
9. NAKATA, I. On compiling algorithms for arithmetic expressions *Comm. ACM 10*, 8 (Aug. 1967), 492–494
10. REDZIEJOWSKI, R R. On arithmetic expressions and trees. *Comm ACM 12*, 2 (Feb. 1969), 81–84
11. SAHNI, S Computationally related problems *SIAM J Comput 3*, 4 (Dec 1974), 262–279
12. SETHI, R. Complete register allocation problems. *SIAM J Comput. 4*, 3 (Sept. 1975), 226–248.
13. SETHI, R , AND ULLMAN, J D. The generation of optimal code for arithmetic expressions. *J. ACM 17*, 4 (Oct 1970), 715–728.
14. SHNEIDER, V. On the number of registers needed to evaluate arithmetic expressions *BIT 11*, 1 (1971), 84–93.
15 STOCKHAUSEN, P.F. Adapting optimal code generation for arithmetic expressions to the instruction sets available on present day computers. *Comm ACM 16*, 6 (June 1973), 353–354 Errata *17*, 10 (Oct 1974), 591.

RECEIVED OCTOBER 1974; REVISED OCTOBER 1975